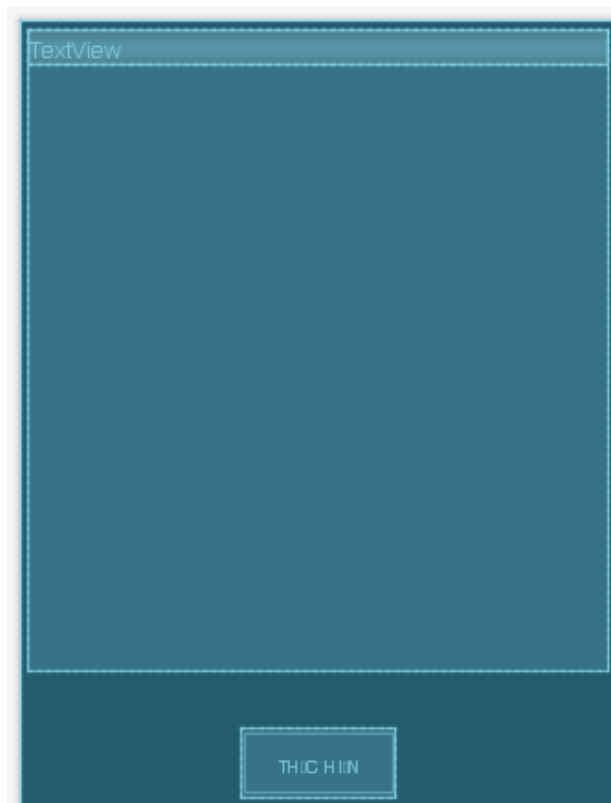


Lab – HTTP

Mô tả: Thực hành truyền, nhận dữ liệu giữa ứng dụng và webservice thông qua HTTP (POST, GET).

1. Tạo 1 project mới đặt tên là DemoHTTP
2. Xây dựng layout gồm 1 scrollView, 1 TextView và 4 Button:



3. Mở file AndroidManifest.xml, bổ sung các dòng được highlight:

```

<uses-permission android:name="android.permission.INTERNET" />
<application
    android:allowBackup="true"
    android:icon="@mipmap/ic_launcher"
    android:label="DemoHTTP"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true"
    android:usesCleartextTraffic="true"
    android:theme="@style/Theme.DemoHTTP">

```

4. Khai báo và tham chiếu đến các đối tượng trên layout:

```

public class MainActivity extends AppCompatActivity {
    TextView textView;
    String kq="";
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        textView=(TextView)findViewById(R.id.textView);
        Button btn_all=(Button)findViewById(R.id.btnAction);
    }
}

```

GET

5. Xây dựng hàm getJsonFromServer:

```

//thực hiện việc request và trả về data dưới dạng chuỗi JSON từ webservice
public static String getJsonFromServer(String url) throws IOException {
    BufferedReader inputStream = null;
    URL jsonUrl = new URL(url);
    URLConnection dc = jsonUrl.openConnection();
    dc.setConnectTimeout(5000);
    dc.setReadTimeout(5000);
    inputStream = new BufferedReader(new InputStreamReader(
        dc.getInputStream()));
    // chuyển dữ liệu Json sang chuỗi
    String jsonResult = inputStream.readLine();
    return jsonResult;
}

```

6. Xây dựng hàm getTheLoai() để lấy tất cả các thể loại trên Webservice:

```

public void getTheLoai(String data)
{
    try {
        JSONArray jsonArray=new JSONArray(data);
        for(int i=0;i<jsonArray.length();i++)
        {
            JSONObject jsonObject=jsonArray.getJSONObject(i);
            kq+=jsonObject.getString( name: "TenTL")+"\t"+jsonObject.getString( name: "ThuTu")+"\n";
        }
    }catch ( JSONException e)
    {
        e.printStackTrace();
    }
    textView.setText(kq);
}

```

7. Tạo class DownLoadJson kế thừa từ đối tượng AsyncTask:

```
private class DownloadJson extends AsyncTask<Void, Void, Void> {
    @Override
    protected Void doInBackground(Void... params) {

        return null;
    }
    @Override
    protected void onPostExecute(Void aVoid) {
        super.onPostExecute(aVoid);
    }
    @Override
    protected void onPreExecute(){
        super.onPreExecute();
    }
}
```

8. Trong class này, bổ sung code cho:

- ***doInBackground:***

Trong đó, url tùy theo webservice mà điều chỉnh lại cho phù hợp.

```
protected Void doInBackground(Void... params) {
    try {
        data = getJsonFromServer( url: "http://10.0.2.2:8080//tintuc/webservices/theloai.php");
    } catch (IOException e) {
        e.printStackTrace();
    }
    return null;
}
```

- ***onPreExecute:***

```

@Override
protected void onPreExecute(){
    super.onPreExecute();
    pd = new ProgressDialog( context: MainActivity.this);
    pd.setTitle("Waiting");
    pd.setMessage("loading....");
    pd.show();
}

```

t

Trong đó, pd là đối tượng tạo từ ProgressDialog được khai báo ở:

```

private class DownloadJson extends Asyn
    ProgressDialog pd; String data="";
    @Override
    protected Void doInBackground(Void.

```

- ***onPostExecute:***

```

@Override
protected void onPostExecute(Void aVoid) {
    super.onPostExecute(aVoid);
    pd.dismiss();
    getTheLoai(data);
}

```

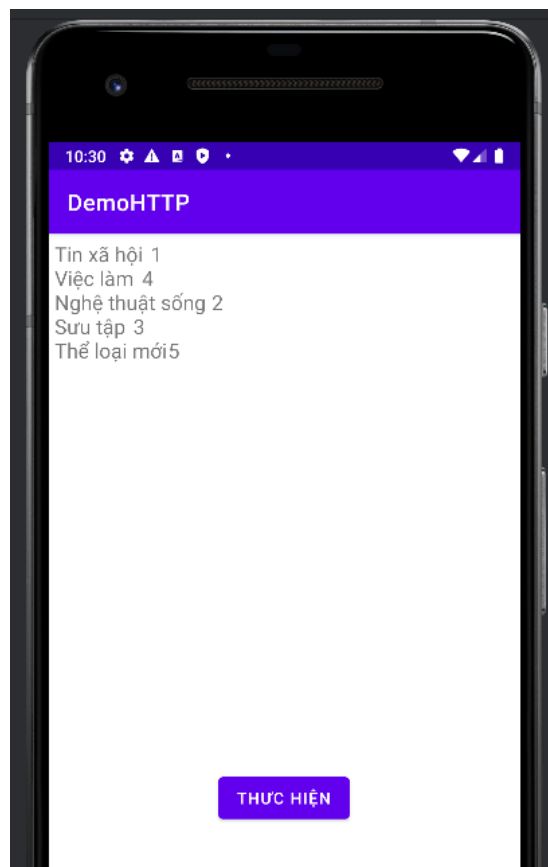
9. Viết code cho sự kiện onClick của btn_all:

```

btn_all.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        DownloadJson downloadJson=new DownloadJson();
        downloadJson.execute();
    }
});

```

10. Chạy project và kiểm tra



11. Xây dựng hàm **getTheLoaiById()** để lấy 1 thể loại theo id cho trước:

```

public void getTheLoaiById(String data) {
    try {
        JSONObject jsonObject = new JSONObject(data);
        kq += jsonObject.getString( name: "TenTL") + "\t" + jsonObject.getString( name: "ThuTu") + "\n";
    } catch (JSONException e)
    {
        e.printStackTrace();
    }
    textView.setText(kq);
}

```

12. Chỉnh lại biến data trong hàm **doInBackground** như hình:

```

data = getJsonFromServer( url: "http://10.0.2.2:8080//tintuc/webservices/theloai.php?id=1");

```

Trong đó, url tùy theo webservice mà điều chỉnh lại cho phù hợp.

13. Trong hàm **onPostExecute**, gọi hàm **getTheLoaiById** thay cho hàm **getTheLoai**

14. Chạy project và kiểm tra kết quả.

POST

15. Xây dựng hàm **postUpdate** để cập nhật dữ liệu của 1 thể loại:

```

public void postUpdate()
{
    try {
        URL url=new URL( spec: "http://10.0.2.2:8080/tintuc/webservices/theloai_update.php");
        HttpURLConnection connection=(HttpURLConnection)url.openConnection();
        connection.setReadTimeout(10000);
        connection.setConnectTimeout(15000);
        connection.setRequestMethod("POST");
        connection.setDoOutput(true);
        connection.setDoInput(true);
        connection.setChunkedStreamingMode(0);
        Uri.Builder builder = new Uri.Builder()
            .appendQueryParameter("id", "8")
            .appendQueryParameter("tentl", "Thế loại mới")
            .appendQueryParameter("thutu", "5")
            .appendQueryParameter("anhien", "0");
        String query = builder.build().getEncodedQuery();
        OutputStream os = connection.getOutputStream();
        BufferedWriter writer = new BufferedWriter(
            new OutputStreamWriter(os, charsetName: "UTF-8"));
        writer.write(query);
        writer.flush();
        writer.close();
        os.close();
        connection.connect();
        int code=connection.getResponseCode();

        System.out.println(code);
    }
    catch (IOException e) { e.printStackTrace(); }
}

```

Lưu ý: dữ liệu dùng cập nhật thể loại tùy thuộc vào dữ liệu thực tế của database mà bạn đang thao tác

16. Gọi hàm postUpdate trong hàm doInBackground:

```

protected Void doInBackground(Void... params) {
    postUpdate();
    return null;
}

```

Và chỉnh lại hàm onPostExecute như sau:

```
@Override
protected void onPostExecute(Void aVoid) {
    super.onPostExecute(aVoid);
    pd.dismiss();
    //...
}
```

17. Chạy project và kiểm tra